

An Analysis of Reinforcement learning human feedback

Reinforcement learning human feedback

Reinforcement learning human feedback -- Part 1

Background and motivation Optimizing a model based on human feedback is desirable when a task is difficult to specify yet easy to judge. For example, one may want to train a model to generate safe text that is both helpful and harmless (such as lacking bias, toxicity, or otherwise harmful content). Asking humans to manually create examples of harmless and harmful text would be difficult and time-consuming. However, humans are adept at swiftly assessing and comparing the harmfulness of different AI-generated text. Therefore, a more practical objective would be to allow the model to use this type of human feedback to improve its text generation. Despite the clear benefits of incorporating human feedback in training models, prior efforts—including some that leverage reinforcement learning (RL)—have encountered significant challenges. Most attempts were either narrow and difficult to generalize, breaking down on more complex tasks, or they faced difficulties learning from sparse (lacking specific information and relating to large amounts of text at a time) or noisy (inconsistently rewarding similar outputs) reward functions. RLHF was not the first successful method of using human feedback for reinforcement learning, but it is one of the most widely used. The foundation for RLHF was introduced as an attempt to create a general algorithm for learning from a practical amount of human feedback. The algorithm as used today was introduced by OpenAI in a paper on enhancing text continuation or summarization based on human feedback, and it began to gain popularity when the same method was reused in their paper on InstructGPT. RLHF has also been shown to improve the robustness of RL agents and their capacity for exploration, which results in an optimization process more adept at handling uncertainty and efficiently exploring its environment in search of the highest reward.

Reinforcement learning human feedback -- Part 2

where $Z(x)$ is the partition function. This is unfortunately not tractable, since it requires summing over all possible responses: $Z(x) = \sum_y \pi_{\text{SFT}}(y|x) \exp(\frac{r(x,y)}{\beta}) = E_{y \sim \pi_{\text{SFT}}(\cdot|x)} [\exp(\frac{r(x,y)}{\beta})]$

$$Z(x) = \sum_y \pi_{\text{SFT}}(y|x) \exp(\frac{r(x,y)}{\beta}) = E_{y \sim \pi_{\text{SFT}}(\cdot|x)} [\exp(\frac{r(x,y)}{\beta})]$$

Reinforcement learning human feedback -- Part 3

Here, β controls how “risk-averse” the value function is (larger β = faster saturation in the logistic function σ) and $z_0 = \text{KL}(\pi_\theta(y'|x) \parallel \pi_{\text{ref}}(y'|x))$

$$z_0 = \text{KL}(\pi_\theta(y'|x) \parallel \pi_{\text{ref}}(y'|x))$$

π_{ref} is a baseline given by the Kullback–Leibler divergence. Since many real-world feedback pipelines yield “like/dislike” data more easily than pairwise comparisons, KTO is designed to be data-efficient and to reflect “loss aversion” more directly by using a straightforward notion of “good vs. bad” at the example level.

Reinforcement learning human feedback -- Part 4

However, instead of doing the intermediate step of the reward model, DPO directly optimizes for the final policy. First, solve directly for the optimal policy, which can be done by Lagrange multipliers, as usual in statistical mechanics: $\pi^*(y|x) = \pi_{\text{SFT}}(y|x) \exp(\frac{r(x,y)}{\beta}) / Z(x)$

$$\pi^*(y|x) = \frac{\pi_{\text{SFT}}(y|x) \exp(\frac{r(x,y)}{\beta})}{Z(x)}$$

Reinforcement learning human feedback -- Part 5

We begin by gathering human preference dataset D . We then fit a reward model r to data, by maximum likelihood estimation using the Plackett–Luce model $r^* = \arg \max_r E_{(x, y_1, \dots, y_N) \sim D} [\ln \prod_{k=1}^N e^{r(x, y_k)} / \sum_{i=1}^N e^{r(x, y_i)}]$

$$r^* = \arg \max_r E_{(x, y_1, \dots, y_N) \sim D} [\ln \prod_{k=1}^N e^{r(x, y_k)} / \sum_{i=1}^N e^{r(x, y_i)}]$$

Reinforcement learning human feedback -- Part 6

Applications RLHF has been applied to various domains of natural language processing (NLP), such as conversational agents, text summarization, and natural language understanding. Ordinary reinforcement learning, in which agents learn from their actions based on a predefined “reward function”, is difficult to apply to NLP tasks because the rewards tend to be difficult to define or measure, especially when dealing with complex tasks that involve human values or preferences. RLHF can steer NLP models, in particular language models, to provide answers that align with human preferences with regard to such tasks by capturing their preferences beforehand in the reward model. This results in a model capable of generating more relevant responses and rejecting inappropriate or irrelevant queries. Some notable examples of RLHF-trained language models are OpenAI’s ChatGPT (and its predecessor InstructGPT), DeepMind’s Sparrow, Google’s Gemini, and Anthropic’s Claude. In computer vision, RLHF has also been used to align text-to-image models. Studies that successfully used RLHF for this goal have noted that the use of KL regularization in RLHF, which aims to prevent the

An Analysis of Reinforcement learning human feedback

Reinforcement learning human feedback

learned policy from straying too far from the unaligned model, helped to stabilize the training process by reducing overfitting to the reward model. The final image outputs from models trained with KL regularization were noted to be of significantly higher quality than those trained without. Other methods tried to incorporate the feedback through more direct training—based on maximizing the reward without the use of reinforcement learning—but conceded that an RLHF-based approach would likely perform better due to the online sample generation used in RLHF during updates as well as the aforementioned KL regularization over the prior model, which mitigates overfitting to the reward function. RLHF was initially applied to other areas, such as the development of video game bots and tasks in simulated robotics. For example, OpenAI and DeepMind trained agents to play Atari games based on human preferences. In classical RL-based training of such bots, the reward function is simply correlated to how well the agent is performing in the game, usually using metrics like the in-game score. In comparison, in RLHF, a human is periodically presented with two clips of the agent's behavior in the game and must decide which one looks better. This approach can teach agents to perform at a competitive level without ever having access to their score. In fact, it was shown that RLHF can sometimes lead to superior performance over RL with score metrics because the human's preferences can contain more useful information than performance-based metrics. The agents achieved strong performance in many of the environments tested, often surpassing human performance.